

Trabajo Práctico – React

Hooks de rendimiento y Contexto en React

1. ¿Qué son los Hooks de rendimiento?

Los Hooks de rendimiento son Hooks de React que permiten optimizar el rendimiento de una aplicación, evitando cálculos innecesarios o actualizaciones que no son necesarias.

Principales Hooks relacionados con rendimiento: **useMemo** memoriza el resultado de un cálculo; **useCallback** memoriza una función; **useTransition** permite marcar ciertas actualizaciones como no urgentes; y **useDeferredValue** permite retrasar la actualización de un valor cuando existe una actualización más importante.

useMemo

useMemo permite guardar en memoria el resultado de una operación y reutilizarlo mientras sus dependencias no cambien.

```
const resultado = useMemo(() => {  
  return operacionCostosa();  
}, [dependencias]);
```

Ejemplo práctico

```
import { useMemo, useState } from "react";  
  
function App() {  
  const [numero, setNumero] = useState(10);  
  const [contador, setContador] = useState(0);  
  
  const resultado = useMemo(() => {  
    console.log("Calculando...");  
    return numero * numero;  
  }, [numero]);  
  
  return (  
    <div>  
      <h1>Resultado: {resultado}</h1>  
  
      <button onClick={() => setNumero(numero + 1)}>  
        Cambiar número  
      </button>  
  
      <button onClick={() => setContador(contador + 1)}>  
        Contador: {contador}  
      </button>  
    </div>  
  );  
}  
  
export default App;
```

Explicación

El componente tiene dos estados: numero y contador. El resultado depende únicamente de numero. Si cambia numero, React vuelve a calcular el resultado. Si cambia solamente contador, React reutiliza el resultado almacenado. Esto resulta útil cuando el cálculo es costoso y queremos evitar repetirlo en cada renderizado.

2. ¿Qué es el Contexto en React?

El Context de React permite compartir información entre diferentes componentes sin tener que pasarla manualmente mediante props por cada nivel de la jerarquía.

Puede utilizarse para compartir información como el usuario autenticado, el tema claro u oscuro, el idioma o configuraciones generales de la aplicación.

El Context se crea normalmente mediante `createContext()` y se consume mediante `useContext()`.

Ejemplo práctico

```
import { createContext, useContext } from "react";

const UsuarioContext = createContext();

function App() {
  const usuario = "Diego";

  return (
    <UsuarioContext.Provider value={usuario}>
      <Usuario />
    </UsuarioContext.Provider>
  );
}

function Usuario() {
  return <Perfil />;
}

function Perfil() {
  const usuario = useContext(UsuarioContext);

  return <h1>Hola {usuario}</h1>;
}

export default App;
```

Explicación

Primero se crea el contexto con `createContext()`. Luego, el componente `App` proporciona el valor mediante `UsuarioContext.Provider`. El componente `Perfil` puede acceder directamente a ese valor mediante `useContext()`, sin recibirlo como prop desde `Usuario`.

Diferencia entre Context y Props

Props	Context
Comunicación principalmente de padre a hijo.	Permite compartir datos entre distintos componentes.
Se pasan explícitamente.	Se accede mediante <code>useContext()</code> .
Útiles para datos específicos de un componente hijo.	Útil para información compartida por varios componentes.
Puede generar prop drilling en estructuras profundas.	Ayuda a evitar el prop drilling.
Hace explícita la dependencia entre componentes.	La dependencia puede quedar menos visible.

Context no reemplaza completamente a las props. Cuando solamente un componente hijo necesita un dato, las props suelen ser una solución más simple y clara.